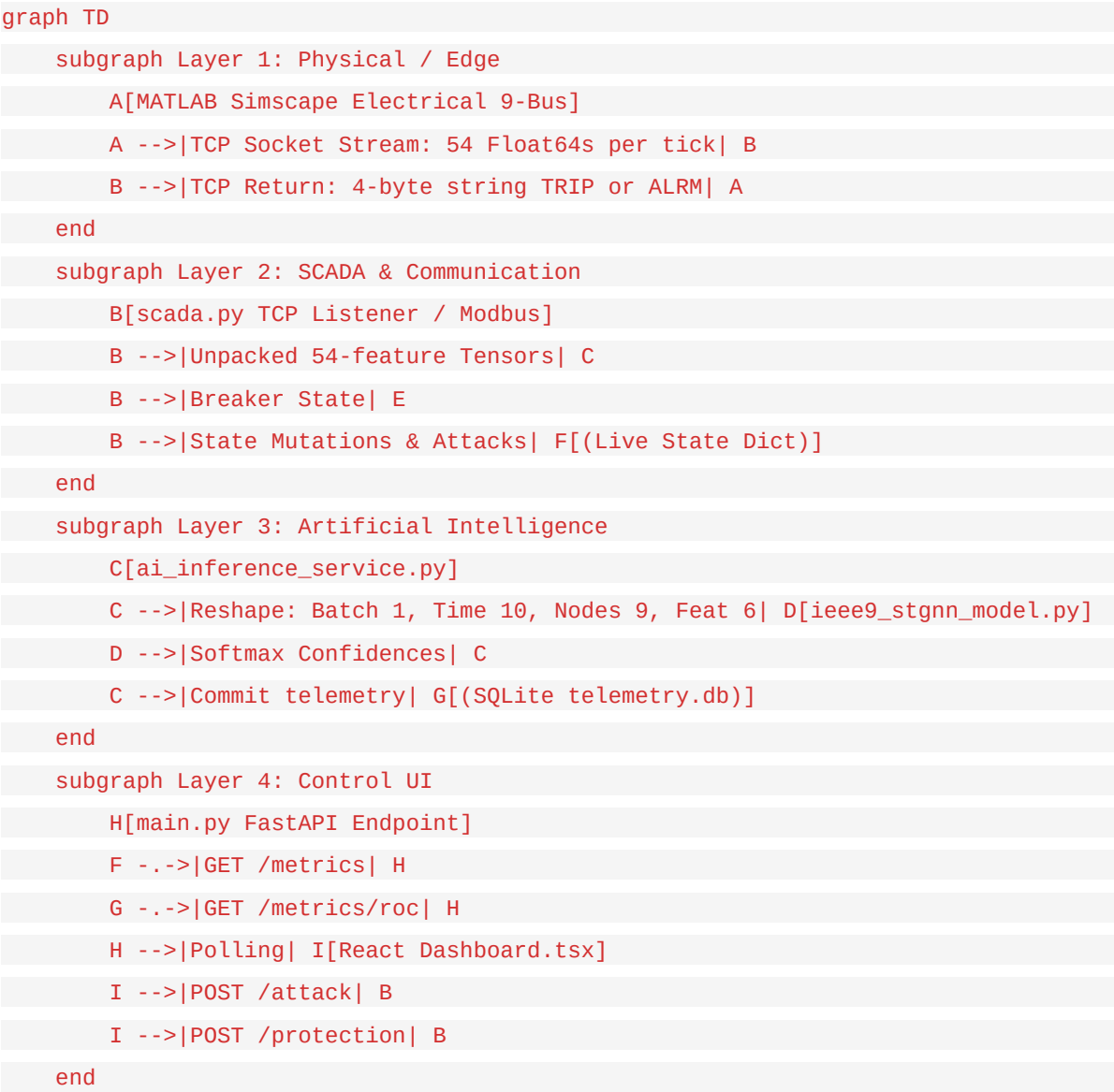


IEEE 9-Bus Cyber-Physical Digital Twin: Comprehensive System Manual

This document is a complete architectural and code-level walkthrough of the **Vertex-Fusion Real-Time AI-Driven Cyber-Physical Digital Twin**. This system bridges a physical electrical grid model (MATLAB Simscape) with an advanced Spatial-Temporal AI detection engine, exposing a closed-loop security dashboard in real-time.

1. Global System Architecture

The project operates universally across 4 distinct technological layers, creating an industry-grade operational relay loop.



■ 2. File-by-File Code Breakdown

■ *Simulation Backend (Python)*

`simulation/scada.py` * **Role:** The primary Data Bridge and SCADA loop handler. * **Key Logic:** * **TCP Listener (`matlab_tcp_listener`):** Spawns a socket on `0.0.0.0:5000`. Crucially, it uses a `recvall` function to rigidly extract exactly **432 bytes** (54 features * 8-byte `float64`) per frame. This strictly prevents TCP fragmentation. * **Fallback Generator (`feeder_to_modbus`):** If MATLAB is disconnected, it natively spins up `ieee9_dynamic_dataset.csv`, extracting 54 columns sequentially to maintain the frontend loop. * **Breaker Logic:** Hard-thresholds at `0.90` (90%) Softmax confidence to shoot the physical **TRIP** command back to the physical layer. Otherwise, acts on an **ALARM** base. * **State Aggregation:** Maps variables symmetrically into the `latest_system_state` dictionary used by the web server.

`simulation/ai_inference_service.py` * **Role:** Execution sandbox for the loaded PyTorch weights. * **Key Logic:** * **Data Reshaping:** Asserts the flat 540-element array directly into the four-dimensional `[1(Batch), 10(Timesteps), 9(Nodes), 6(Features)]` structure that spatial computation requires. * **Probabilistic Distribution:** Instead of just parsing binary labels, it derives the exact Softmax decimal metrics against [FDI, DoS, Replay] vector probabilities. * **Telemetry Logging:** Invokes `_log_to_db` strictly appending (`timestamp`, `true_label`, `pred_label`, `confidence`, `latency_ms`) to trace the machine learning ground truths necessary for dynamic GUI evaluation calculations.

`simulation/stgnn_model.py` & `simulation/ieee9_stgnn_model.py` * **Role:** The mathematical architecture of the AI protection. * **Key Logic:** * **Topological Array (`edge_index`):** Directly maps to the physical transmission wires built into the IEEE 9-bus benchmark (e.g., node 4 connects to nodes 1, and 5). * **GATConv Layers:** Instead of classical 1D ML convolutions, the model performs actual message passing across the specified `edge_index`. Voltages dropping at Bus 4 physically affect computations at Bus 5 directly through Edge Attention weights. * **LSTM Aggregation:** Flattens the 32 spatial features of the 9 buses across 10 temporal windows to identify waveform timing anomalies (like Replay attacks).

`simulation/main.py` * **Role:** The Python FastAPI router defining HTTP accessibility. * **Routes:** * **GET /metrics:** Spits out the complete array dict of all 9 active buses processed by `scada.py`. * **GET /metrics/roc:** Connects to `telemetry.db`, fetching the last 100 historical records so the frontend mathematical parser can deduce analytical Area Under Curve algorithms. * **POST /attack:** Accepts user manual overrides from the dashboard to poison the input stream tensor prior to inference scaling.

`simulation/test_matlab_tcp.py` * **Role:** Utility script mimicking actual MATLAB Simulink hardware. * **Key Logic:** Streams 54 randomized mock voltages/currents to port 5000 packed using `np.tobytes()`. Enforces a rigid 4-byte socket listen (`sock.recv(4)`) to assert correct **TRIP** extraction framing.

`simulation/periodic_retrainer.py` * **Role:** Offline Academic script proving continuous learning safety bounds. * **Key Logic:** Grabs confident predictions from the `.db`, initializes Backpropagation natively without choking the sub-5ms latency bounds of the actual SCADA hardware script, outputs a pristine `.pth` copy ready for deployment.

■ ■ Frontend Visualization (React / TSX)

`frontend/src/pages/Dashboard.tsx` * **Role:** The central Cyber-Physical Command & Interactive user interface. * **Key Logic:** * **Polling Structure:** Spins up `useCallback` on `setInterval` tracking `poll()` (realtime metrics) and `pollRoc()` (evaluation telemetry) every 500ms universally. * **Grid Mathematics:** * `avgVoltage`: Derived dynamically by calculating the mean of nodes 1-9 sequentially. * **ROC Integral Computations:** * Extracts `preds` and `trues` to calculate `numPositive/numNegative`. * Populates a step staircase coordinate mapping True Positive Rates (`TPR`) against False Positive Rates (`FPR`). * Runs an analytical mathematical Riemann integral across the FPR gap boundaries to constantly stream an accurate real-time `AUC` decimal score in the application header. * **Recharts Plotting:** Embeds standard React D3/Recharts arrays tracking 9 individual `<Line>` attributes matching standard dashboard telemetry standards.

■ 3. Feature Mapping Table (IEEE 9-Bus System)

This represents the exact feature indices pushed securely through the TCP connection block.

[Table Data]

Bus No. | Features Exported per node (Double / Float64 Array)

Bus 1 | [V_mag, V_ang, P, Q, Freq, I_mag]

Bus 2 | [V_mag, V_ang, P, Q, Freq, I_mag]

Bus 3 | [V_mag, V_ang, P, Q, Freq, I_mag]

... | ...

Bus 9 | [V_mag, V_ang, P, Q, Freq, I_mag]

Total Features per Cycle: 54 parameters

Sliding Window Span: 10 cycles

Tensor Payload Processed: 540 physical parameters per AI decision tick.

[!TIP] The strict IEEE topological bounding requires `[V_mag, V_ang, P, Q, Freq, I_mag]` to be standardized. Changes to this order will destroy the GATConv spatial relationships and necessitate model retraining.

■ 4. Run Sequence Checklist

To operate the true digital twin at scale: 1. **Start the Python Engine:** `python simulation/main.py`. This exposes `http://127.0.0.1:8000` endpoints and stands up the TCP listener wrapper mapping port `5000`. 2. (Optional but Required for Full HIL Loop) **Run the Physical Sim:** Trigger `backend/matlab/tcp_server_demo.m` (or native test script) to fire cyclic telemetry blocks directly down to the listener. 3. **Launch Dashboard:** Boot the Vite client (`npm run dev`). 4. **Inject Cyber Threats:** Observe the mathematical disruption via the FDI / DoS buttons. Note the `AUC` dropping linearly during unseen attack distributions.

This concludes the cyber-physical digital twin systems breakdown.